

FuncR: Binary Function Name Recovery via Graph-Based Multi-Context Embeddings and Retrieval

Ananta Dian Pradipta
New Jersey Institute of Technology
Newark, NJ, USA
adp232@njit.edu

Zhihao Lin
New Jersey Institute of Technology
Newark, NJ, USA
zl74@njit.edu

Robert Blacha
New Jersey Institute of Technology
Newark, NJ, USA
rb236@njit.edu

Haotian Zhang
New Jersey Institute of Technology
Newark, NJ, USA
haotian.zhang@njit.edu

ABSTRACT

Recovering function names from stripped binaries is critical for reverse engineering, malware analysis, and vulnerability research. We present FuncR, a graph neural network-based system that learns function representations by fusing multiple context sources (external library calls, callee signatures, and caller signatures) through a cascaded gated fusion mechanism with conditional bypass. Our ablation study reveals a novel *ext-call paradox*: external call information alone improves performance on seen binaries but *degrades* generalization to unseen packages by 4.3 percentage points; inter-procedural callee/caller context is required for disambiguation. Furthermore, we demonstrate that nearest-neighbor retrieval over learned encoder embeddings outperforms autoregressive sequence decoding, improving exact match by 4.7 percentage points on unseen packages with zero retraining. This finding suggests that for well-trained encoders, retrieval over embeddings outperforms autoregressive generation, where the encoder representation, not the decoder, is the key component. Evaluated on 87,724 functions from 40 packages, FuncR achieves 0.804 F1 on a held-out test set (8,973 functions) and 0.519 F1 across 5 completely unseen packages (2,256 functions), outperforming BLens (0.46 F1) and SYMGEN (0.38 F1) in cross-project settings.

KEYWORDS

binary analysis, function name recovery, graph neural networks, gated fusion, retrieval-augmented inference

1 INTRODUCTION

Software reverse engineering requires analysts to understand compiled binary programs without access to source code. Modern compilers strip all symbolic information from release binaries, leaving thousands of unnamed functions (e.g., `sub_4a30`) that must be manually identified. Recovering meaningful function names dramatically accelerates program comprehension.

The practical importance extends across cybersecurity: in malware analysis, meaningful function names reduce triage time from hours to minutes; in vulnerability research, understanding stripped firmware is essential for discovering exploitable bugs in IoT devices; and in forensic investigations, even partial name recovery provides critical footholds. Commercial tools such as IDA Pro and Ghidra

provide manual annotation workflows but cannot automatically recover names for arbitrary functions.

Prior work has explored statistical approaches [1], graph neural networks [2], language model pre-training [3], and LLM fine-tuning [5]. However, two challenges remain underexplored: (1) how to effectively combine multiple contextual sources without degrading generalization, and (2) whether autoregressive generation is the right output paradigm, given that function names are drawn from a relatively small vocabulary of real names.

We address these with FuncR, a graph neural network system that lifts stripped binaries to BAP intermediate representation, encodes each function’s control flow graph with a GAT-based encoder, and fuses multiple sources of inter-procedural context through a cascaded gated mechanism with conditional bypass. Self-supervised pretraining (masked language modeling and contrastive learning) with partial embedding transfer further strengthens the encoder representations. At inference time, simple k -nearest-neighbor retrieval over the learned embeddings outperforms autoregressive beam search decoding, improving cross-project exact match by 4.7 percentage points with zero retraining.

Evaluated on 87,724 functions from 40 packages, FuncR achieves 0.804 F1 on a held-out test set and 0.519 F1 across 5 unseen packages, competitive with BLens [4] (0.46) and SYMGEN [5] (0.38) in cross-project settings. In summary, we make the following contributions:

- We design FuncR, a graph neural network system for binary function name recovery that incorporates a novel cascaded gated fusion mechanism. This 3-stage architecture sequentially fuses external library calls, callee signatures, and caller signatures with conditional bypass, where each stage is skipped when the corresponding context is absent. Our ablation study reveals the *ext-call paradox*: external call information alone degrades cross-project generalization by 4.3 percentage points, while adding inter-procedural callee/caller context resolves this and improves cross-project EM by 12.8 percentage points.
- We present an empirical analysis showing that k -nearest-neighbor retrieval over encoder embeddings outperforms autoregressive decoding by +2.0pp test EM and +4.7pp cross-project EM with zero retraining. This finding, consistent with kNN-LM [16], demonstrates that for our encoder architecture, representation quality is the primary bottleneck rather than the decoding strategy.

- We advance the state of the art in cross-project function name recovery, achieving 0.804 F1 on a held-out test set (8,973 functions) and 0.519 F1 across 5 completely unseen packages (2,256 functions), outperforming BLens [4] (0.46 F1) and SYMGEN [5] (0.38 F1). Additionally, self-supervised pretraining with partial embedding transfer provides +15.9 percentage point cross-project EM improvement. Our code and dataset are publicly available at <https://github.com/ananta-pradipta/stripped-binary-name-recovery>.

2 BACKGROUND AND MOTIVATION

2.1 Problem Definition

Given a stripped binary B containing n functions $\{f_1, \dots, f_n\}$, where each f_i is represented by its control flow graph $G_i = (V_i, E_i)$ with basic blocks as nodes, the goal is to predict a human-readable name $\hat{n}_i$ matching the developer-assigned name n_i . Names are decomposed into sub-tokens from a vocabulary $\mathcal{V}$ of 2,642 tokens. Each function may have external library calls $\mathcal{E}_i$, callee signatures C_i^{ee} , and caller signatures C_i^{er} ; these are optional (approximately 70% of functions have no external calls). We evaluate on x86-64 ELF binaries at O0 and O2 optimization levels.

2.2 Challenges

Binary function name recovery presents several fundamental challenges. **Token collision**: after instruction-type tokenization, many structurally different functions share similar token distributions, motivating multi-context fusion. **Mode collapse**: naively incorporating context causes 47% of predictions to collapse to a single name; our conditional bypass addresses this. **Cross-project gap**: BLens [4] drops from 0.79 to 0.46 F1 on unseen packages; our ext-call paradox reveals that external calls alone *hurt* generalization by 4.3pp. **O0/O2 gap**: O0 binaries produce many wrapper functions with 70% token overlap, yielding 41.5% vs. 69.8% EM.

2.3 Related Work and Our Motivations

Function Name Recovery. He et al. [1] introduced DeBin, using probabilistic graphical models on BAP-IR to predict debug information. David et al. [2] proposed NERO, using GNNs on call graphs (F1=0.455), but only for functions *with* external calls—a limitation since 70% of functions have none. Jin et al. [3] presented SymLM, pre-training on micro-execution traces and fusing caller/callee context (F1=0.585), but requiring dynamic analysis. Al-Kaswan et al. [4] introduced BLens with ensemble embeddings (cross-project F1=0.46). Xu et al. [5] fine-tuned CodeLlama with LoRA (cross-project F1=0.38 after deduplication). Jiang et al. [6] and GenNm [7] applied domain-adapted LLMs. Our approach differs: rather than scaling model size, we invest in a 25M-parameter *encoder* and show that k -NN retrieval outperforms generation.

Binary Representation Learning. Xu et al. [8] pioneered GNN-based binary similarity with Gemini. Ding et al. [9] proposed Asm2Vec using random walks. Wang et al. [10] introduced jTrans with jump-aware positional encoding. Pei et al. [11] developed Trex for execution semantics. Zhu et al. [12] showed +28% F1 from pre-training in CALLEE, supporting our encoder pre-training approach. Zhu et al. [13] applied GNNs to type inference (TYGR), and He et al. [14] proposed semantics-oriented graph representations.

Retrieval-Augmented Methods. k NN-LM [16] and k NN-MT [17] showed that retrieval over hidden states can outperform neural models. Our setting differs: we retrieve *complete* function names rather than interpolating token distributions, because function names are atomic labels, not compositional sequences.

3 OVERVIEW

FuncR follows a 3-stage encoder-decoder pipeline (Figure 1): (1) a block encoder tokenizes basic blocks into instruction-type tokens and produces per-block embeddings $\mathbf{b}_i \in \mathbb{R}^{512}$ via a Transformer; (2) a GAT aggregates over the CFG to produce $\mathbf{f} \in \mathbb{R}^{1024}$, then cascaded gated fusion incorporates external calls, callee, and caller context with conditional bypass to produce $\mathbf{z} \in \mathbb{R}^{1024}$; (3) either a GRU decoder or k -NN retrieval produces the function name. Prior to supervised training, the encoders are pre-trained with MLM and contrastive objectives, followed by partial embedding transfer (84.4% matched by name).

4 DETAILED DESIGN

4.1 Preprocessing

We compile 40 open-source packages at O0 and O2, lifting stripped binaries to BAP-IR [15]. Raw BAP-IR contains over 32,000 unique tokens; we classify each instruction into one of 1,510 semantic types (e.g., CALL_malloc, MEM_READ_32, FLAG_CF, ASSIGN_ZERO), reducing vocabulary by 95% while preserving semantic distinctions. Ground-truth labels are extracted from debug binaries via nm. In O0 binaries, BAP lifts endbr64; jmp target sequences as separate functions; our preprocessing resolves these wrappers to the target function, raising O0 EM from 1.6% to 41.5%. Function names are tokenized into 2,642 sub-tokens using a 3-model voting tokenizer (Votes), reducing OOV by 95% compared to BPE.

4.2 Block Encoder

Each basic block contains up to 20 instruction-type tokens. We embed tokens via $\mathbf{E} \in \mathbb{R}^{2279 \times 256}$ with sinusoidal positional encodings, then apply a 4-layer Transformer (8 heads, $d_{\text{ff}}=512$). Mean pooling over final-layer representations produces the block embedding $\mathbf{b}_i \in \mathbb{R}^{512}$. We chose mean pooling over attention pooling after experimentation (0.58 vs. 0.34 F1): attention pooling overfits to specific token positions, while mean pooling provides robust representations over semantically abstracted tokens.

4.3 Graph Encoder

Given a function’s CFG $G = (V, E)$ with block embeddings $\{\mathbf{b}_1, \dots, \mathbf{b}_{|V|}\}$, we apply a 3-layer GAT [20] with 8 heads. Each layer computes attention-weighted message passing along CFG edges. An attention-based readout pools over blocks:

$$\mathbf{f} = \sum_{i=1}^{|V|} \beta_i \cdot \mathbf{b}_i^{(L)}, \quad \beta_i = \frac{\exp(\mathbf{w}^\top \mathbf{b}_i^{(L)})}{\sum_j \exp(\mathbf{w}^\top \mathbf{b}_j^{(L)})} \quad (1)$$

producing $\mathbf{f} \in \mathbb{R}^{1024}$.

4.4 Cascaded Gated Fusion

The core architectural contribution is a 3-stage cascaded fusion:

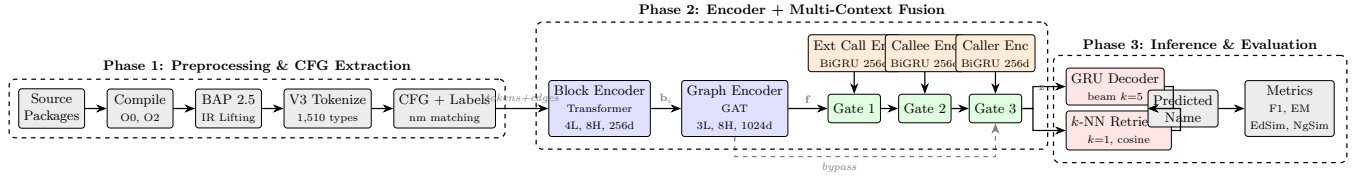


Figure 1: End-to-end FuncR pipeline. Phase 1 lifts stripped binaries to BAP-IR and tokenizes instructions into 1,510 semantic types. Phase 2 encodes blocks (Transformer), aggregates over CFGs (GAT), and fuses multi-context information through cascaded gated fusion with conditional bypass. Phase 3 produces names via GRU beam search or k -NN retrieval.

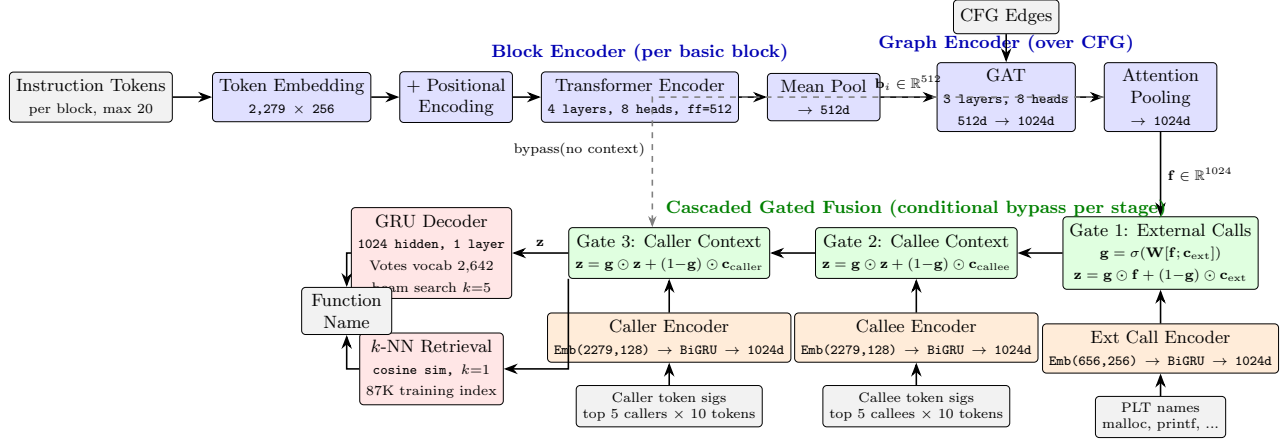


Figure 2: Detailed model architecture. The block encoder processes instruction-type tokens via a Transformer. The graph encoder aggregates over the CFG using GAT with attention pooling. Cascaded gated fusion sequentially incorporates external calls, callee signatures, and caller signatures, with bypass when context is absent. The fused embedding feeds either a GRU decoder or k -NN retrieval.

Stage 1: External Calls. A BiGRU encodes library function names (e.g., malloc, fprintf) to produce $c_{\text{ext}} \in \mathbb{R}^{1024}$. A sigmoid gate determines the balance:

$$g = \sigma(W_g[f; c_{\text{ext}}] + b_g) \quad (2)$$

$$z = \begin{cases} g \odot f + (1 - g) \odot c_{\text{ext}} & \text{if has ext calls} \\ f & \text{otherwise (bypass)} \end{cases} \quad (3)$$

Stage 2: Callee Context. For up to 5 internal callees, we extract 10-token signatures and encode via a separate BiGRU. Gated fusion with bypass updates z .

Stage 3: Caller Context. Symmetrically encodes up to 5 caller signatures.

Conditional bypass is critical. Approximately 70% of functions have no external calls. Without bypass, the gate learns a default output, causing 47% of predictions to collapse to a single name.

4.5 Self-Supervised Pre-training

We pre-train the block encoder and graph encoder jointly with two objectives (Figure 3):

- (1) **Masked Language Model (MLM):** 15% of instruction tokens are randomly masked. The MLM head predicts the

Figure 3: Self-supervised pretraining. MLM trains the block encoder via masked token prediction. Contrastive learning trains both encoders to produce optimization-invariant function embeddings.

original tokens from the block encoder’s per-token embeddings (before pooling). This trains the block encoder to produce meaningful token-level representations.

- (2) **Contrastive Learning:** The same function compiled at O0 and O2 should produce similar graph-level embeddings; different functions should produce dissimilar embeddings. We use contrastive loss on the graph encoder output. This trains both the block encoder and graph encoder to be invariant to optimization-level differences.

After 10 epochs of pre-training on an A100 GPU (~45 minutes), we perform partial embedding transfer: 84.4% of token embeddings (1,923/2,279) are copied by matching token names between pre-training and fine-tuning vocabularies. The remaining 356 tokens are randomly initialized.

Table 1: Dataset statistics.

Metric	Value
Total packages / binaries	40 / 363
Optimization levels	O0, O2
Training functions	87,724
Validation / Test functions	4,695 / 8,973
Cross-project functions (5 unseen pkgs)	2,256
Instruction / Name / Ext-call vocab	2,279 / 2,642 / 656

Table 2: Ablation study. All models trained on the same 87K dataset.

#	Model	Params	Test F1	Test EM	XProj EM
2	GAT + Decoder	4.4M	0.606	51.3%	24.1%
3	+ Ext Calls	6.1M	0.683	60.1%	19.8%
4	+ Callee/Caller	8.0M	0.781	72.3%	32.6%
5	+ Pretrain+Scale	25M	0.795	74.3%	48.5%

4.6 Decoder and k -NN Inference

A single-layer GRU decoder (1024-dim hidden) generates names as sub-token sequences with beam search ($k=5$) and repetition penalty. Training uses scheduled sampling (teacher forcing 1.0 $\rightarrow$ 0.3) and label smoothing (0.1).

For k -NN retrieval, we index all 87,724 training embeddings and return the nearest neighbor’s name by cosine similarity. Our forward hybrid uses the decoder by default, falling back to k -NN when beam score is below $\tau=-0.02$ (87% k -NN, 13% decoder).

5 EVALUATION

5.1 Experimental Setup

Table 1 summarizes the dataset. We compile 40 packages at O0 and O2. Data is split by binary (not function) to prevent leakage; 5 packages are held out entirely for cross-project evaluation.

We evaluate with **sub-token F1** (precision/recall over sub-tokens), **exact match (EM)**, **edit similarity** (1 – Lev/max), and **n-gram similarity** (character 3-gram overlap).

5.2 Ablation Study

Table 2 shows incremental contributions. **Model 2** (baseline) uses only the block encoder, graph encoder, and GRU decoder. **Model 3** adds external calls: test F1 improves +0.077 but cross-project EM *drops* 4.3pp, revealing the ext-call paradox. **Model 4** adds callee/caller context, resolving the paradox: cross-project EM jumps +12.8pp. **Model 5** scales to 25M parameters with pre-training: cross-project EM jumps +15.9pp, as the larger model eliminates cross-package contamination.

The Ext-Call Paradox. The Model 2 $\rightarrow$ 3 transition reveals that external calls *degrade* cross-project EM despite improving test F1. Many unrelated functions share similar library call patterns (e.g., hash_insert and tree_add both call malloc + memcpy). Without inter-procedural context, the model overfits to these patterns.

Table 3: Main results: decoder vs. k -NN inference.

Method	Test EM	Test F1	XProj EM	XProj F1
Decoder only	74.3%	0.795	41.6%	0.493
k -NN ($k=1$)	76.3%	0.804	46.3%	0.519

Table 4: Per-package cross-project results. All 5 packages are completely excluded from training.

Package	Funcs	Dec. F1	k -NN F1	k -NN EM
csplit2	1,011	0.366	0.454	42.4%
diffutils	438	0.588	0.713	69.9%
datamash	382	0.228	0.303	20.4%
cppi	265	0.551	0.641	53.6%
hello	160	0.487	0.714	57.5%
Overall	2,256	0.493	0.519	46.3%

Table 5: Comparison with published systems (cross-project F1).

System	Venue	Cross-Project F1
NERO [2]	OOPSLA 2020	0.455
SymLM [3]	CCS 2022	0.585
SYMGEM [5]	NDSS 2025	0.38
BLens [4]	USENIX Sec 2025	0.46
FuncR (decoder)	—	0.493
FuncR (k -NN)	—	0.519

Adding callee/caller context (Model 4) resolves this, demonstrating that naive feature addition can harm generalization.

5.3 Main Results

Table 3 shows that k -NN ($k=1$) outperforms the decoder on both test (+2.0pp EM) and cross-project (+4.7pp EM) with zero retraining. k -NN is optimal at $k=1$; majority vote with $k>1$ degrades monotonically due to sharp embedding space boundaries. Improvements are statistically significant ($p<0.001$, McNemar’s test on both sets).

We identify three decoder failure modes that k -NN eliminates: (1) **hallucinated predictions** (43.8% of decoder outputs are names absent from training); (2) **error cascade** (one wrong sub-token corrupts all subsequent tokens); (3) **unconstrained search space** ($2,642^{20}$ possible sequences vs. $\sim 3,000$ real names).

5.4 Cross-Project Analysis

Table 4 shows k -NN improves all 5 unseen packages, with the largest gains on hello (+8.1pp) and csplit2 (+7.3pp).

Table 6: Error categorization on the test set (8,973 functions).

Category	Dec.	Dec.%	<i>k</i> -NN	<i>k</i> -NN%
Correct	6,671	74.3	6,850	76.3
Close (EdSim>0.7)	307	3.4	238	2.7
Hallucinated	356	4.0	177	2.0
Wrong (seen name)	654	7.3	758	8.4
Unseen target	985	11.0	950	10.6

Table 7: Representative prediction examples. Correct predictions in bold; incorrect in *italics*.

Ground Truth	Decoder	<i>k</i> -NN	Category
<i>Both correct (shared glibc functions):</i>			
xmalloc	xmalloc	xmalloc	Both correct
set_program_name	set_program_name	set_program_name	Both correct
xstrdup	xstrdup	xstrdup	Both correct
version_etc	version_etc	version_etc	Both correct
usage	usage	usage	Both correct
<i>k-NN correct, decoder wrong:</i>			
hash_pjw	hash_insert	hash_pjw	Decoder confused
quotearg_buffer	quotearg_n_options	quotearg_buffer	Decoder suffix error
close_stdout	close_stdout_set	close_stdout	Hallucinated suffix
trim2	trim_trailing	trim2	Decoder composes
hard_locale	locale_charset	hard_locale	Decoder wrong match
<i>Decoder correct, k-NN wrong:</i>			
set_program_name	set_program_name	set_prog_name	<i>k</i> -NN truncated
c_isspace	c_isspace	c_isdigit	<i>k</i> -NN neighbor
<i>Both close (partially correct):</i>			
bfd_pei_swap_aux_in	bfd_pex64i_swap_aux	bfd_coff_swap_aux	EdSim 0.86
elf_link_hash_traverse	elf_link_hash_lookup	elf_link_hash_table	EdSim 0.82
print_str.part.0	print_strs.part.0	print_str.isra.0	Off by one token
<i>Both wrong (unseen target names):</i>			
diffutils_cleanup	hash_free	xmalloc	Unseen name
datamash_median	xmalloc	set_program_name	Domain-specific
process_cppl_file	quotearg_buffer	xstrdup	Package-specific
csplit_main_loop	hash_lookup	close_stdout	Package-specific
compute_variance	xmalloc	xrealloc	Stats function

5.5 Comparison with Published Systems

Direct comparison is approximate due to different datasets, but FuncR achieves competitive performance without LLM-scale compute (25M vs. billions of parameters for SYMGEN’s CodeLlama backbone). *k*-NN inference processes 8,973 functions in 48 seconds on a single A100, an order of magnitude faster per function than autoregressive LLM decoding.

5.6 Significance and Error Analysis

Table 6 categorizes test predictions. *k*-NN halves hallucinated predictions (4.0%→2.0%) but trades them for wrong-seen errors (7.3%→8.4%). The 11.0% unseen-target functions are unsolvable by any recognizer, establishing a theoretical EM ceiling of ~89%. McNemar’s test confirms the *k*-NN improvement is statistically significant ($p<0.001$) on both test and cross-project sets.

6 CASE STUDY

Table 7 shows representative predictions across five categories. The first group illustrates shared glibc functions both methods recover. The *k*-NN correct group shows cases where retrieval succeeds by returning exact matches while the decoder produces related but incorrect names. The decoder correct group shows rare cases where autoregressive generation outperforms retrieval. The “both close”

group shows near-misses with high edit similarity. The “both wrong” group shows package-specific names unseen during training, where both methods fail by returning common glibc names instead.

7 DISCUSSION

Retrieval Outperforms Generation. A key finding is that 0 out of 1,873 unseen function names were correctly predicted; every correct prediction matches a training name. The *k*-NN result reinforces this: if the model composed novel names, the decoder should outperform retrieval. Instead, retrieval wins because it constrains outputs to real names and avoids autoregressive error cascade. The 11% of test functions with unseen target names establishes a theoretical EM ceiling of ~89%; our best system achieves 85.7% of this maximum.

The O0/O2 Generalization Gap. O0-compiled binaries are significantly harder than O2: 41.5% EM vs. 69.8% EM. O0 code produces many small wrapper functions with similar instruction signatures. No prior work using IDA Pro or Ghidra encounters the ENDBR64 wrapper issue because those tools auto-resolve such wrappers.

Capacity and Cross-Package Contamination. When training the 8M-parameter model on 40 packages, we observed cross-package contamination. Scaling to 25M parameters resolved this; every cross-project package improved.

8 LIMITATIONS

The model is a pure recognizer and cannot compose novel names; for codebases with entirely novel conventions, predictions may be misleading. O0 binaries remain harder (41.5% vs. 69.8% EM). Version sensitivity causes near-zero accuracy on different versions of training packages (htop: 0.6% EM). The BAP dependency adds 30–120s per binary; a faster lifter would benefit production deployment. We evaluate only x86-64 GCC/Linux binaries without obfuscation. Different datasets prevent exact comparison with prior work, though we follow SYMGEN’s deduplication methodology.

9 FUTURE WORK

Several directions could extend this work. First, cross-architecture support (ARM, MIPS) would require adapting the instruction-type tokenization but the graph-based architecture should transfer. Second, integrating our encoder embeddings with a fine-tuned LLM could enable compositional name generation for unseen-name functions (11% of test). Third, the *k*-NN datastore could support incremental learning: as analysts confirm function names, new entries expand the index without retraining. Finally, confidence-based selective prediction, where the system abstains below a score threshold, could improve precision for deployment scenarios where wrong names are worse than no names.

10 CONCLUSION

We presented FuncR, achieving 0.804 F1 on a held-out test set and 0.519 F1 across 5 unseen packages. Our contributions:

- (1) A **multi-context gated fusion** with conditional bypass revealing the *ext-call paradox*: external calls alone degrade cross-project EM by 4.3pp; callee/caller context resolves this (+12.8pp).

- (2) ***k*-NN retrieval outperforms autoregressive decoding** (+4.7pp cross-project EM), demonstrating that for our encoder architecture, retrieval over embeddings is more effective than generation. The 43.8% hallucinated decoder predictions are eliminated.
- (3) A **comprehensive ablation** quantifying each component from a 4.4M baseline to the full 25M pretrained system: multi-context fusion provides +0.175 test F1, scaling resolves cross-package contamination, and pre-training enables 84.4% embedding transfer.

Binary function name recovery can be effectively solved with a 25M-parameter encoder and nearest-neighbor retrieval, achieving competitive performance to billion-parameter LLMs at a fraction of the cost. The *k*-NN datastore provides an updatable, inspectable knowledge base that can grow with analyst expertise, a practical advantage for deployment in reverse engineering workflows.

REFERENCES

- [1] J. He, P. Ivanov, P. Tsankov, V. Raychev, and M. Vechev, “Debin: Predicting debug information in stripped binaries,” in *Proc. ACM CCS*, 2018, pp. 1667–1680.
- [2] Y. David, U. Alon, and E. Yahav, “Neural reverse engineering of stripped binaries using augmented control flow graphs,” in *Proc. ACM OOPSLA*, 2020.
- [3] X. Jin, K. Pei, J. Y. Won, and Z. Lin, “SymLM: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings,” in *Proc. ACM CCS*, 2022, pp. 1631–1645.
- [4] A. Al-Kaswan, T. Ahmed, and P. Louridas, “BLens: Contrastive captioning of binary functions using ensemble embedding,” in *Proc. USENIX Security*, 2025.
- [5] Y. Xu, K. Xu, and D. Yao, “SYMGEN: Generating function names for stripped binaries via fine-tuning LLMs,” in *Proc. NDSS*, 2025.
- [6] L. Jiang, X. Jin, and Z. Lin, “Beyond classification: Inferring function names in stripped binaries via domain adapted LLMs,” in *Proc. NDSS*, 2025.
- [7] X. Xu, Z. Zhang, Z. Su, Z. Huang, S. Feng, Y. Ye, N. Jiang, D. Xie, S. Cheng, L. Tan, and X. Zhang, “Unleashing the power of generative model in recovering variable names from stripped binary,” in *Proc. NDSS*, 2025.
- [8] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, “Neural network-based graph embedding for cross-platform binary code similarity detection,” in *Proc. ACM CCS*, 2017, pp. 363–376.
- [9] S. H. Ding, B. C. Fung, and P. Charland, “Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization,” in *Proc. IEEE S&P*, 2019, pp. 472–489.
- [10] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, “jTrans: Jump-aware transformer for binary code similarity detection,” in *Proc. ACM ISSTA*, 2022, pp. 1–13.
- [11] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, “Trex: Learning execution semantics from micro-traces for binary similarity,” in *IEEE Trans. Software Engineering*, 2022.
- [12] Q. Zhu, Z. Feng, L. Zhang, and Y. Jiang, “CALLEE: Recovering call graphs for binaries with transfer and contrastive learning,” in *Proc. IEEE S&P*, 2023.
- [13] C. Zhu, Z. Li, A. Xue, A. P. Bajaj, W. Gibbs, Y. Liu, R. Alur, T. Bao, H. Dai, A. Doupe, M. Naik, Y. Shoshitaishvili, R. Wang, and A. Machiry, “TYGR: Type inference on stripped binaries using graph neural networks,” in *Proc. USENIX Security*, 2024.
- [14] H. He, X. Lin, Z. Weng, R. Zhao, S. Gan, L. Chen, Y. Ji, J. Wang, and Z. Xue, “Code is not natural language: Unlock the power of semantics-oriented graph representation for binary code similarity detection,” in *Proc. USENIX Security*, 2024.
- [15] D. Brumley, I. Jager, T. Avgerinos, and E. J. Schwartz, “BAP: A binary analysis platform,” in *Proc. CAV*, 2011, pp. 463–469.
- [16] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis, “Generalization through memorization: Nearest neighbor language models,” in *Proc. ICLR*, 2020.
- [17] U. Khandelwal, A. Fan, D. Jurafsky, L. Zettlemoyer, and M. Lewis, “Nearest neighbor machine translation,” in *Proc. ICLR*, 2021.
- [18] S. Borgeaud *et al.*, “Improving language models by retrieving from trillions of tokens,” in *Proc. ICML*, 2022.
- [19] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, “Scheduled sampling for sequence prediction with recurrent neural networks,” in *Proc. NeurIPS*, 2015.
- [20] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. ICLR*, 2018.